```
  "synopsis" : "A must-have book for every Java programmer and Java, ...",
  "edition" : 3,
  "amazon_rating" : 4.7,
  "release_date" : "2017-12-27",
  ...
}
```

When we index the document for the first time, because we do not create an index up front, Elasticsearch deduces the schema by analyzing the values of the fields. The `edition` field, for example, is represented as a numeric (non-text) field and hence is derived as a `long` data type by the engine.

Using similar logic, the `amazon_rating` field is determined to be a `float` data type because the field includes decimal values. Finally, the `release_date` is determined to be a `date` data type because the value is represented in ISO8601 date format (`yyyy-MM-dd`). All three fields are categorized as *non-text* fields, meaning the values are not tokenized (not split into tokens) and not normalized (no synonyms or no root words) but are stored as they are.

Term-level queries produce binary output: results are fetched if the query matches the criteria; otherwise, no results are sent. These queries do not consider *how well the documents match (relevancy)*; instead, they concentrate on whether the query has a match. As relevancy is not considered, term-level queries do not produce a relevancy score. Let's look at a few term-level queries in the next two sections.

2.4.1 The term query

A `term` query is used to fetch exact matches for a value provided in the search criteria. For example, to fetch all third edition books, we can write a `term` query as the next listing shows.

Listing 2.16 Fetching third edition books

```
GET books/_search
{
  "_source": ["title","edition"],
  "query": {
    "term": {
      "edition": {
        "value": 3
      }
    }
  }
}
```

We return just two fields in the response document.

Declares the query as a term-level query

Provides the field and value as search criteria

This query returns all third edition books (our index includes only one: *Effective Java*), as shown here:

```
"hits" : [{
  ...
  "_score" : 1.0,
  "_source" : {
```